

セキュリティ実践演習第一 第2回 マルウェア解析

理工学部理工学科

知能情報システムプログラム

DX人材育成基盤プログラム

池部実 minoru@oita-u.ac.jp

マルウェア解析についての演習

- マルウェア解析(1) 表層解析
- マルウェア解析(2) 逆アセンブリ
- マルウェア解析(3) 動的解析
- マルウェア解析(4) コードからの特徴抽出
- マルウェア解析(5) 機械学習・深層学習によるマルウェア分類
- マルウェア対策・攻撃検知

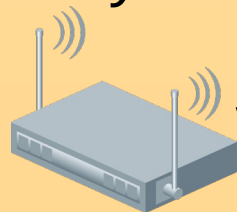
演習内容

- Pythonを用いてバイナリファイルを解析
- 機械学習などを用いてマルウェアを分類
- 演習を踏まえて、マルウェア対策・攻撃検知のための方法を検討する

演習環境(1)

example.jp.
10.0.0.0/24

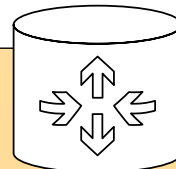
SSID: SecurityLab
(hidden)



Ubuntu
DHCP
10.0.0.0/24



ルータ(NAPT)



binary.example.jp.
10.0.0.160/24
(Ubuntu Server)

準備(1)

- 演習端末(DELL, Ubuntu)
 - login id: security
 - password: *****(Moodle上資料では非公開)
 - 端末に記載
 - 無線LAN: SecurityLab 接続(済)
- 演習用NWは, B-Coreで演習端末からのみ接続可
- 時間外に演習用サーバに接続する場合, 踏み台サーバを利用する(後述)

準備(2)

- サーバには, Ubuntu(ノートPC)のVSCodeから `ssh(extension)` で接続
- 接続先サーバ: `binary.example.jp`.
- サーバには, `security` ユーザでログイン
 - パスワードはUbuntu(ノートPC)と同じ
- `binary.example.jp`上で実行したウィンドウを手元のUbuntu(ノート)で表示したい場合,
`ssh -X binary.example.jp`
`ssh -Y binary.example.jp`
としてSSH接続する

~/ .ssh/config

- binary.example.jpで実行したmatplotlibのウィンドウをUbuntu(ノートPC)上に表示
- ~/ .ssh/configに以下のように記述

```
Host binary.example.jp
  HostName binary.example.jp
  User security
  ForwardX11 yes
  ForwardX11Trusted yes
```

(注意事項)Ubuntu上でのpip

- UbuntuなどのLinuxや, macOSでは共通環境へのpipインストールは推奨されていない
- ユーザ毎にvirtualenvの利用を推奨

```
python3 -m pip install matplotlib  
error: externally-managed-environment
```

```
× This environment is externally managed  
↳ To install Python packages system-wide, try apt install  
python3-xyz, where xyz is the package you are trying to  
install.
```

If you wish to install a non-Debian-packaged Python package, create a virtual environment using `python3 -m venv path/to/venv`. Then use `path/to/venv/bin/python` and `path/to/venv/bin/pip`. Make sure you have `python3-full` installed.

If you wish to install a non-Debian packaged Python application, it may be easiest to use `pipx install xyz`, which will manage a virtual environment for you. Make sure you have `pipx` installed.

See `/usr/share/doc/python3.12/README.venv` for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or OS, by passing `--break-system-packages`.

hint: See PEP 668 for the detailed specification.

binary.example.jp

- sshでログインすると, venv環境を自動で読み込むように設定
 - (入力の必要なし) `source ~/.venv/bin/activate`
 - プロンプトの先頭に(.venv)が付いていればOK
- pipでインストールの際
 - `$ python3 -m pip MODULENAME`
- Binary Refineryはpipでインストール済み

演習内容

- Jesko Hüttenhain氏開発のバイナリファイルを解析するためのPythonベースのツール
Binary Refinery
 - <https://binref.github.io>
- Windows, Linux, macOSなど様々なOSでしよう可能
- バイナリデータの表示, 切り出し, デコードなどのコマンドを提供
 - コマンドをパイプ(|)でつなげて実行してバイナリファイルを解析可能
- Binary Refineryを用いて難読化コードを分析

バイナリ解析レポート

- 今回の演習では、`binary.example.jp`上でのターミナルでの実行結果をすべてテキストファイルに記載しておくこと

Binary Refinery 入出力機能

- 入出力に関するコマンド
 - ef (emit file)
 - 引数で指定したファイルの内容を標準出力へ出力
 - pf (print format)
 - フォーマットを指定してパイプで別のコマンドから受け取ったデータやそのデータに関する情報を出力
 - emit
 - dump

efコマンド使用例

```
$ echo 0123 > 0123.txt
```

```
$ ef 0123.txt  
0123
```

catコマンドと同様の機能

```
$ cat 0123.txt  
0123
```

pfコマンド使用例

- cfmtで使用可能なフォーマット
 - {}: パイプで受け取ったデータ自体
 - {size}: データのサイズ
 - {ext}: データから自動判別したファイル拡張子
 - {entropy}: データのエントロピー(0-1)
 - {ic}: データの偏りを表す指数(0-1) (一致指数)
 - 例) AAAAの偏りは, Aのみから構成されているので1.0
 - 例) ABCDの偏りは, 異なる文字で構成されており0.0
 - {magic}: データから自動判別したファイルの種類
 - {crc32}: 32ビットのCRCチェックサム
 - {md5}: MD5ハッシュ値
 - {sha1}: SHA-1ハッシュ値
 - {sha256}: SHA-256ハッシュ値

pfコマンド使用例

// \$以降の青文字は1行で入力

```
$ ef 0123.txt | pf "Size:{size}  
Extension:{ext} Type:{magic} Content:  
{}"
```

```
Size:5 Extension:txt Type:ASCII text  
Content: 0123
```

[| コマンド] 記法

- 複数のファイルの内容のデータそれぞれに対して指定したコマンドを実行可能
- Binary Refineryが提供するコマンドを指定する必要
 - UNIXコマンド不可
- [|  cmd ] のようにcmd前後に空白文字(で表現)を入れる
- pfコマンドでフォーマットにファイルの相対パスを表す{path}を利用可能

pfコマンドの使用例

[| コマンド] 記法を用いた場合

```
$ echo abcd > abcd.txt
```

```
$ ef abcd.txt
```

```
abcd
```

// \$以降の青文字は1行で入力

```
$ ef abcd.txt 0123.txt [ | pf "Path:{path},  
Size:{size}, Type:{magic}, Content:{}" ]
```

```
Path:abcd.txt, Size:5, Type:ASCII text,  
Content:abcd
```

```
Path:0123.txt, Size:5, Type:ASCII text,  
Content:0123
```

catとefの違い

- catコマンドで複数ファイルを引数に指定した場合、標準出力に複数ファイルを結合して1つのファイルとして扱うため、| (パイプ) では受け取った出力全体にコマンドを適用することになる
- efコマンドで複数ファイルを引数に指定して、[| cmd] の記法と組み合わせると、それぞれのファイルに対してcmdを適用可能

emitコマンド使用例

- emitコマンド
 - UNIXのechoコマンドのように引数に指定した文字列を出力するコマンド
 - h:16真数
 - q:URL (エンコードされた文字列)
 - b64:Base64エンコードされた文字列

emitコマンド使用例

```
$ emit h:507974686f6e
```

Python

```
$ emit
```

```
q:%E5%AE%9F%E8%B7%B5%E3%81%99%E3%82%8B
```

実践する

```
$ emit b64:440Q44Kk440K440q6Kej5p6Q
```

バイナリ解析

(参考)

文字列⇒16進数: Pythonでは, `b"Python".hex()` (※ASCII文字のみ)

URLエンコード: <https://www.tagindex.com/tool/url.html>

BASE64エンコード: <https://www.en-pc.jp/tech/base64/>

dumpコマンド使用例

- dumpコマンド
 - パイプで受け取ったデータを引数で指定されたファイル名のファイルに保存
- 使用例
 - 文字列“Binary Refinery”を16進数にエンコードしたものをemitコマンドでデコードして，dumpコマンドでファイルへ保存
 - pfコマンドで{md5}.{ext}と指定することで受け取ったデータのMD5を求め，ファイルの拡張子txtにもとづくファイル名で保存

dumpコマンド使用例

```
$ emit h:42696e61727920526566696e657279
```

```
Binary Refinery
```

```
// $以降の青文字は1行で入力
```

```
$ emit h:42696e61727920526566696e657279  
| dump {md5}.{ext}
```

```
$ ls
```

```
0123.txt
```

```
0788af3c85056c668c8ef9a7661e58dd.txt
```

```
abcd.txt
```

MD5のハッシュ値

(参考)

文字列⇒16進数: Pythonでは, b" Binary Refinery".hex() (※ASCII文字のみ)

Binary Refineryデータの 切り出し機能

- データの切り出し機能に関するコマンド
 - snip
 - chop

peekコマンド使用例

- peekコマンド
 - ファイルの内容を16進数で出力(ダンプ)
 - 類似コマンドにLinuxのhexdumpコマンドやxxdコマンドがあるが, peekコマンドではファイルのサイズ, エントロピーや自動判別したファイルの種類の情報も表示可能
 - デフォルト: ターミナルの幅に合わせて表示するデータのバイト数が増減し, 10行分のデータを表示
 - オプション-W 16オプションで1行に16バイトずつ表示
 - オプション-a で, ファイルの内容すべて出力
 - オプション-m で, サイズ等の情報を複数行で出力

peekコマンド使用例

```
$ ef /bin/ls | peek -W 16
```

```
-----  
00.142 MB; 74.19% entropy; ELF 64-bit LSB shared object, x86-64, vers...
```

```
-----  
00000: 7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00 .ELF.....  
00010: 03 00 3E 00 01 00 00 00 30 6D 00 00 00 00 00 00 ..>.....0m.....  
00020: 40 00 00 00 00 00 00 00 28 24 02 00 00 00 00 00 @.....($.....  
00030: 00 00 00 00 40 00 38 00 0D 00 40 00 1F 00 1E 00 ....@.8...@.....  
00040: 06 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00 .....@.....  
00050: 40 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 @.....@.....  
00060: D8 02 00 00 00 00 00 00 D8 02 00 00 00 00 00 00 .....  
00070: 08 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00 .....  
00080: 18 03 00 00 00 00 00 00 18 03 00 00 00 00 00 00 .....  
00090: 18 03 00 00 00 00 00 00 1C 00 00 00 00 00 00 00 .....  
-----
```

00: グレー

09(TAB), 0A(LF), 0D(CR), 20(SPACE): 青

21-7E(ASCII印字可能文字): ターミナル標準の文字色(例: 黒)

それ以外の文字: 赤

peekコマンド使用例

```
$ ef /bin/ls | peek -W 10 -m
```

```
-----  
entropy = 74.19%
```

```
magic = ELF 64-bit LSB shared object, x86...
```

```
size = 00.142 MB  
-----
```

```
00000: 7F 45 4C 46 02 01 01 00 00 00 .ELF.....  
0000A: 00 00 00 00 00 00 03 00 3E 00 .....>..  
00014: 01 00 00 00 30 6D 00 00 00 00 ....0m....  
0001E: 00 00 40 00 00 00 00 00 00 00 ..@.....  
00028: 28 24 02 00 00 00 00 00 00 00 ($.....  
00032: 00 00 40 00 38 00 0D 00 40 00 ..@.8..@..  
0003C: 1F 00 1E 00 06 00 00 00 04 00 .....  
00046: 00 00 40 00 00 00 00 00 00 00 ..@.....  
00050: 40 00 00 00 00 00 00 00 40 00 @.....@..  
0005A: 00 00 00 00 00 00 D8 02 00 00 .....  
-----
```

snipコマンド使用例

- snipコマンド
 - パイプで受けとったデータを範囲を指定して切り出す
 - 範囲指定 始点:終点:ステップ
 - Pythonのスライスと同様の記法
 - 始点: nを指定すると, n+1バイト目
(先頭0起点として考えるが, 1バイト目から数える)
 - オプション-r
 - サイズ等の情報を表示しない

snipコマンド使用例

```
$ emit h:00112233445566778899aabbccddeeff | peek -W 16 -r
```

```
-----  
00: 00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF  .."3DUfw.....
```

```
-----  
$ emit h:00112233445566778899aabbccddeeff | snip 4:12 | peek -W 16 -r
```

```
-----  
0: 44 55 66 77 88 99 AA BB                                DUfw....
```

```
-----  
$ emit h:00112233445566778899aabbccddeeff | snip :-5:-1 | peek -W 16 -r
```

```
-----  
0: FF EE DD CC                                           ....
```

```
-----  
$ emit h:00112233445566778899aabbccddeeff | snip ::2 | peek -W 16 -r
```

```
-----  
0: 00 22 44 66 88 AA CC EE                                ."Df....
```

snipコマンド使用例(解説)

- `snip 4:12`
 - 5バイト目から12バイト目まで
- `snip :-5:-1`
 - 最後のバイトから逆順(-1)で最後から4バイト目まで
- `snip ::2`
 - 1バイトおきに最初のバイトから最後から2バイト目まで

chopコマンド使用例

- chopコマンド
 - パイプで受けとったデータを引数で指定したバイト数ごとに分割
 - [`|cmd`]記法を使って分割したデータにpeekコマンドを実行して表示可能
 - chop 8
 - 8バイトごとに分割
 - chop 5
 - 5バイトごとに分割

chopコマンド使用例

```
$ emit h:00112233445566778899aabbccddeeff | chop 8 [| peek -W 16 -r ]
```

```
-----  
0: 00 11 22 33 44 55 66 77                .."3DUfw
```

```
-----  
0: 88 99 AA BB CC DD EE FF                .....
```

```
$ emit h:00112233445566778899aabbccddeeff | chop 5 [| peek -W 16 -r ]
```

```
-----  
0: 00 11 22 33 44                .."3D
```

```
-----  
0: 55 66 77 88 99                Ufw..
```

```
-----  
0: AA BB CC DD EE                .....
```

```
-----  
0: FF                .
```

Binary Refineryバイナリと数値の変換機能

- バイナリと数値の変換機能に関するコマンド
 - hex
 - pack

hexコマンド使用例

- hexコマンド
 - パイプで受けとった16進数でエンコードされたデータを元のデータにデコード
 - エンコードされたデータはコンマ区切りやスペース区切りの形式でもデコード可能
 - オプション-R
 - データを16進数でエンコード(逆の動作)

hexコマンド使用例

```
$ emit 00112233445566778899aabbccddeeff | hex | peek -W 16 -r
```

```
-----  
00: 00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF  .."3DUfw.....  
-----
```

```
$ emit "Binary Refinery" | hex -R
```

```
42696E61727920526566696E657279
```

```
$ emit 42696E61727920526566696E657279 | hex
```

```
Binary Refinery
```

```
$ emit 42 69 6E 61 72 79 20 52 65 66 69 6E 65 72 79 | hex
```

```
Binary Refinery
```

packコマンド使用例

- packコマンド
 - hexコマンドよりも汎用的で、引数に基数を指定することで16進数に加えて2進数、8進数や10進数などについてもデコード可能

packコマンド使用例

```
$ emit 66,105,110,97,114,121,32,82,101,102,105,110,101,114,121 | pack
```

Binary Refinery

```
$ emit "Binary Refinery" | pack -R
```

66

105

110

97

114

121

32

82

101

102

105

110

101

114

121

```
$ emit "Binary Refinery" | pack -R [| cca , ]
```

66,105,110,97,114,121,32,82,101,102,105,110,101,114,121,

packコマンド使用例

```
$ emit h:01020304 | pack -R 2
```

```
1
```

```
10
```

```
11
```

```
100
```

```
$ emit h:01020304 | pack -R 2 -w 8
```

```
00000001
```

```
00000010
```

```
00000011
```

```
00000100
```

```
$ emit h:01020304 | pack -R 2 -w 8 [| cca " " ]
```

```
00000001 00000010 00000011 00000100
```

```
$ emit h:07080910 | pack -R 8 -w 8 [| cca " " ]
```

```
00000007 00000010 00000011 00000020
```

```
$ emit h:07080910 | pack -R 16 -w 8 [| cca " " ]
```

```
00000007 00000008 00000009 00000010
```

Binary Refineryビット演算

- ビット演算機能に関するコマンド
 - add
 - sub
 - neg
 - rotl
 - rotr
 - shl
 - shr

addコマンド使用例

- addコマンド
 - パイプで受けとったデータの各バイトに
コマンドの引数(10進数)で指定された値を加算
 - 引数で指定する値のprefixに各文字列を付与
 - 0bで2進数
 - 0oで8進数
 - 0xあるいはh:で16進数
 - h:バイト列 を引数に指定可能
 - 2つのバイト列の同じ位置のバイト同士を加算
 - h: ビッグエンディアン
 - 0x: リトルエンディアン

addコマンド使用例

```
$ emit h:0011223344556677 | peek -W 16 -r
```

```
-----  
0: 00 11 22 33 44 55 66 77                .."3DUfw  
-----
```

// バイト列の加算(各バイトに8を加算)

```
$ emit h:0011223344556677 | add 8 | peek -W 16 -r
```

```
-----  
0: 08 19 2A 3B 4C 5D 6E 7F                ..*;L]n.  
-----
```


addコマンド使用例

// バイト列に16進数で11223344の4バイトのバイト列を加算

```
$ emit h:0011223344556677 | add h:11223344 | peek -W 16 -r
```

```
-----  
0: 11 33 55 77 55 77 99 BB .3UwUw..  
-----
```

(解説)

- 0011223344556677よりも11223344のほうが短い
ため、先頭から11223344を00112233に加算してい
き、44556677にはまた11223344を加算している
- add h:11223344なので、ビッグエンディアン表現

addコマンド使用例

// バイト列に16進数で11223344の4バイト(リトルエンディアン)のバイト列を加算

// 0x44332211で, ビッグエンディアン11223344と同様の表記

```
$ emit h:0011223344556677 | add 0x44332211 | peek -W 16 -r
```

0: 11 33 55 77 55 77 99 BB .3UwUw..

// バイト列に16進数で11223344の4バイト(リトルエンディアン)のバイト列を加算

// h:44332211 の加算

```
$ emit h:0011223344556677 | add 0x11223344 | peek -W 16 -r
```

0: 44 44 44 44 88 88 88 88 DDDD.....

subコマンド使用例

- subコマンド
 - パイプで受けとったデータの各バイトに
コマンドの引数(10進数)で指定された値を減算
 - 引数で指定する値のprefixに各文字列を付与
 - 0bで2進数
 - 0oで8進数
 - 0xあるいはh:で16進数
 - h:バイト列 を引数に指定可能
 - 2つのバイト列の同じ位置のバイト同士を加算
 - h: ビッグエンディアン
 - 0x: リトルエンディアン

subコマンド使用例

// バイト列の減算(各バイトから8を減算)

```
$ emit h:0011223344556677 | sub 8 | peek -W 16 -r
```

0: F8 09 1A 2B 3C 4D 5E 6F

...+<M^o

subコマンド使用例

// バイト列に16進数で11223344の4バイトのバイト列を減算

```
$ emit h:0011223344556677 | sub h:11223344 | peek -W 16 -r
```

```
-----  
0: EF EF EF EF 33 33 33 33                      ....3333  
-----
```

(解説)

- 0011223344556677よりも11223344のほうが短い
ため、先頭から11223344から00112233に減算して
いき、44556677にはまた11223344を減算している
- sub h:11223344なので、ビッグエンディアン表現

subコマンド使用例

```
// バイト列に16進数で11223344の4バイト(リトルエンディアン)のバイト列を減算
```

```
// 0x44332211で, ビッグエンディアン11223344と同様の表記
```

```
$ emit h:0011223344556677 | sub 0x44332211 | peek -W 16 -r
```

```
-----  
0: EF EE EE EE 33 33 33 33          ....3333  
-----
```

```
// バイト列に16進数で11223344の4バイト(リトルエンディアン)のバイト列を減算
```

```
// h:44332211 の減算
```

```
$ emit h:0011223344556677 | sub 0x11223344 | peek -W 16 -r
```

```
-----  
0: BC DD FF 21 00 22 44 66          ...!. "Df  
-----
```

negコマンド使用例

- negコマンド
 - ビット反転
 - 引数なし

// 11001001をpack 2で2進数でデコードして,
その結果をビット反転し, 2進数にエンコード

```
$ emit 11001001 | pack 2 | neg | pack -  
R 2 -w 8
```

00110110

シフト演算

- 論理シフト
- 算術シフト
- 循環シフト(ローテート)
- 違いは？
- ここでは、論理シフトと循環シフトを取りあげる

rot1コマンド使用例

- rot1コマンド

- ビットの左へのローテート(循環シフト)
- 引数は, ローテートするビット数

// 11001001をpack 2でデコードしてその結果を左に3ビットローテートし, 2進数にエンコード

```
$ emit 11001001 | pack 2 | rot1 3 | pack -R 2  
-w 8
```

01001110

- 11001001 を左に3ビットローテート(循環シフト)
 - 10010011 (左に1ビットローテート)
 - 00100111 (左に2ビットローテート)
 - 01001110 (左に3ビットローテート)

rotrコマンド使用例

- rotrコマンド

- ビットの右へのローテート(循環シフト)
- 引数は, ローテートするビット数

// 11001001をpack 2でデコードしてその結果を右に3ビットローテートし, 2進数にエンコード

```
$ emit 11001001 | pack 2 | rotr 3 | pack -R  
2 -w 8
```

00111001

- 11001001 を右に3ビットローテート(循環シフト)
 - 11100100 (右に1ビットローテート)
 - 01110010 (右に2ビットローテート)
 - 00111001 (右に3ビットローテート)

shlコマンド使用例

- shlコマンド

- ビットの左への論理シフト
- 引数は、シフトするビット数

// 11001001をpack 2でデコードしてその結果を左に3ビット論理シフトし、2進数にエンコード

```
$ emit 11001001 | pack 2 | shl 3 | pack -R 2  
-w 8
```

01001000

- 11001001 を左に3ビット論理シフト
 - 10010010 (左に1ビット論理シフト)
 - 00100100 (左に2ビット論理シフト)
 - 01001000 (左に3ビット論理シフト)

shrコマンド使用例

- shrコマンド

- ビットの右への論理シフト
- 引数は、シフトするビット数

// 11001001をpack 2でデコードしてその結果を右に3ビット論理シフトし、2進数にエンコード

```
$ emit 11001001 | pack 2 | shr 3 | pack -R 2  
-w 8
```

00011001

- 11001001 を右に3ビット論理シフト
 - 01100100 (右に1ビット論理シフト)
 - 00110010 (右に2ビット論理シフト)
 - 00011001 (右に3ビット論理シフト)

複数バイトでのシフト演算

- デフォルトではバイトの境界をまたがず、各バイトに閉じた形でシフト演算
- 例) 10000000 00010000 00001000 00000001
を5ビット左にローテート

```
$ emit 10000000 00010000 00001000 00000001  
| pack 2 | rotl 5 | pack -R 2 -w 8
```

00010000

00000010

00000001

00100000

複数バイトでのシフト演算

- オプション-Bでブロックサイズを指定することで、ブロックサイズ内の範囲内でバイトの境界をまたいでシフト演算可能
 - デフォルト: リトルエンディアン(-Eでビッグエンディアン)
- 例) 10000000 00010000 00001000 00000001 を5ビット左にローテート(2バイトブロック内)

```
$ emit 10000000 00010000 00001000 00000001 |  
pack 2 | rotl -B 2 5 | pack -R 2 -w 8
```

00000010

00010000

00000000

00100001

10000000 00010000 00001000 00000001

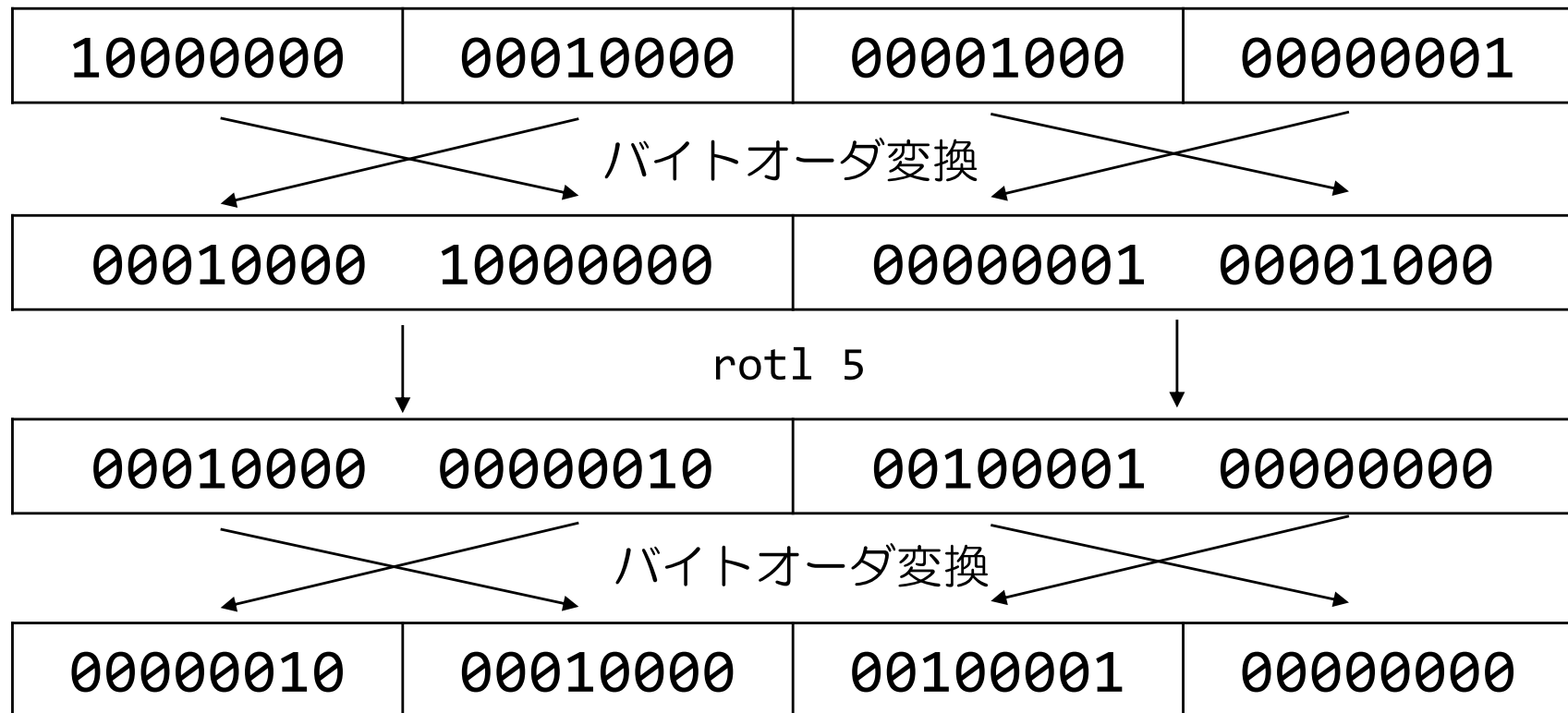
00000010 00010000 00000000 00100001

2バイトの範囲内で
左に5ビットローテート

2バイトの範囲内で
左に5ビットローテート

-B オプション時のバイトオーダー

- -Bオプション指定時は、デフォルトでバイト列をリトルエンディアンとして扱うため、ローテート前後でバイトオーダーを変換



論理シフト

- 左に5ビット論理シフト

```
$ emit 10000000 00010000 00001000 00000001  
| pack 2 | shl 5 | pack -R 2 -w 8
```

00000000

00000000

00000000

00100000

論理シフト

- 左に5ビット論理シフト(2バイトの範囲内でシフト)

```
$ emit 10000000 00010000 00001000 00000001  
| pack 2 | shl -B 2 5 | pack -R 2 -w 8
```

00000000

00010000

00000000

00100001

10000000 00010000 00001000 00000001
00000000 00010000 00000000 00100001

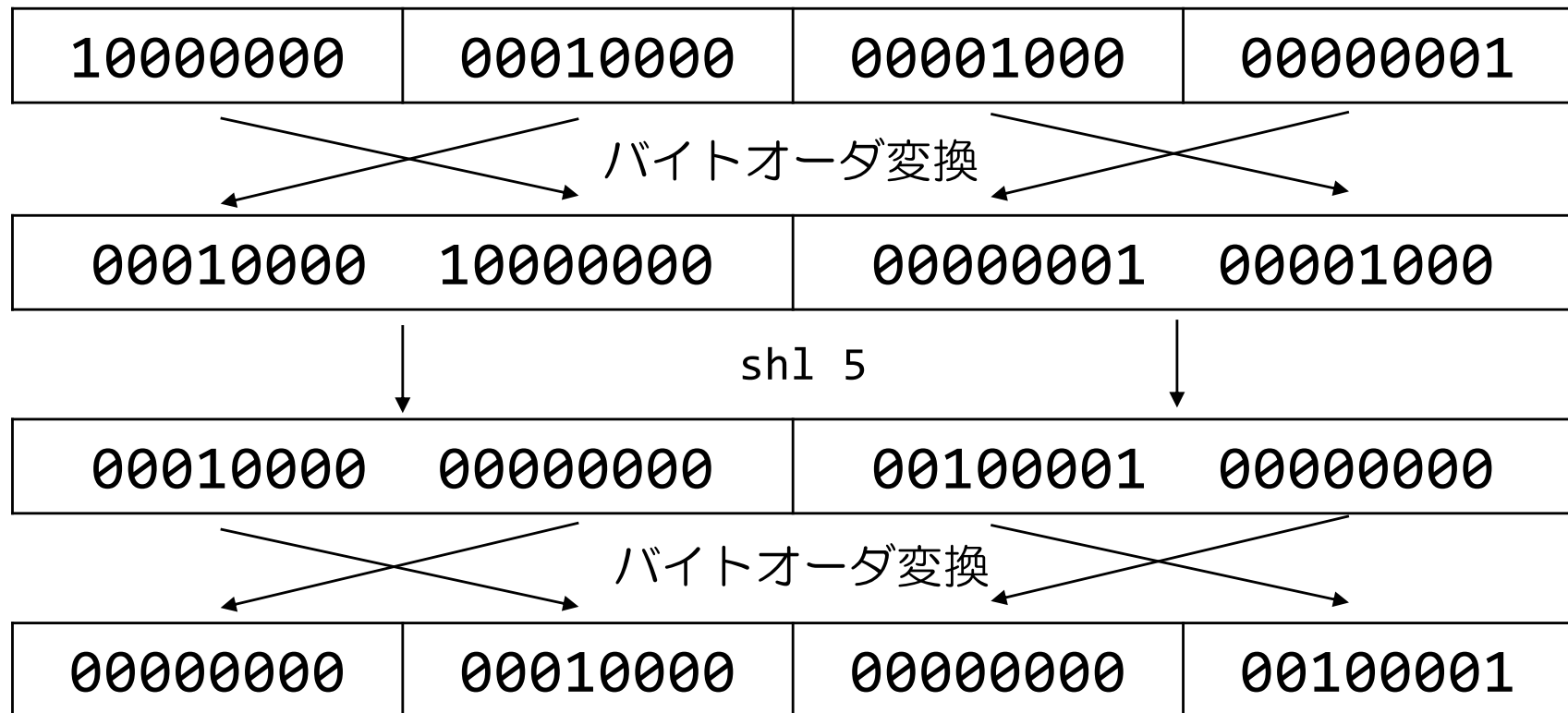
2バイトの範囲内で
左に5ビットローテート

2バイトの範囲内で
左に5ビットローテート

※ -B オプション指定時はリトルエンディアンとして扱う
ためシフトの前後で、バイトオーダを変換している

-B オプション時のバイトオーダ

- -Bオプション指定時は、デフォルトでバイト列をリトルエンディアンとして扱うため、ローテート前後でバイトオーダを変換



Binary Refinery XOR演算

- XOR(排他的論理和)演算機能

- xorコマンド
- xkeyコマンド
- autoxorコマンド

$A \oplus B$		
A	B	$A \oplus B$
True	True	False
True	False	True
False	True	True
False	False	False

- 排他的論理和(eXclusive OR; XOR)

- 2つの命題を組み合わせたとき, 命題A, Bのいずれか片方のみが成立するときに真(True)となる論理演算

xorコマンド使用例

- xorコマンド
 - パイプで受けとったデータの各バイトに対して引数に指定された値(XORキー)を用いてXORを演算
 - XORキーには複数バイト分の値を指定可能
- 文字a(01100001, 0x61)に対してXORキー0x23(00100011)でXOR演算

```
$ emit h:61 | xor h:23 | pack -R 2 -w 8
```

```
01000010
```

- 0x61: 01100001
- 0x23: 00100011
- xor 01000010

XOR演算の使用方法

- マルウェアの通信先ホスト名などの設定情報や、追加で感染させる別のマルウェアのファイルなどのマルウェア内部に埋め込まれているデータを解析者がマルウェアのファイル内の文字列を抽出したり実行ファイルのヘッダのシグネチャで検索しても発見できないように、マルウェア開発者が隠すための簡易的な手法としてXOR演算を利用する
- 1バイトのXORキーであれば、16進数で01からFFまでの値でXOR演算を総当たりで試行して文字列や実行ファイルのヘッダなどがもっともらしい内容になっているかどうかを確認してXORキーを特定

XORキーの推定

- xkeyコマンド
 - XOR演算後のデータをパイプで受けとり，データから推定したXORキーを出力
- 例) /bin/lsファイルに対してXORキー0xCCを使ってXOR演算した結果の先頭部分をpeekコマンドで表示

```
$ ef /bin/ls | xor 0xcc | peek -W 16 -r
```

```
-----  
00000: B3 89 80 8A CE CD CD CC CC CC CC CC CC CC CC .....  
00010: CF CC F2 CC CD CC CC CC FC A1 CC CC CC CC CC .....  
00020: 8C CC CC CC CC CC CC CC E4 E8 CE CC CC CC CC .....  
00030: CC CC CC CC 8C CC F4 CC C1 CC 8C CC D3 CC D2 CC .....  
00040: CA CC CC CC C8 CC CC CC 8C CC CC CC CC CC CC .....  
00050: 8C CC CC CC CC CC CC CC 8C CC CC CC CC CC CC .....  
00060: 14 CE CC CC CC CC CC CC 14 CE CC CC CC CC CC .....  
00070: C4 CC CC CC CC CC CC CC CF CC CC CC C8 CC CC CC .....  
00080: D4 CF CC CC CC CC CC CC D4 CF CC CC CC CC CC .....  
00090: D4 CF CC CC CC CC CC CC D0 CC CC CC CC CC CC .....  
-----
```

XORキーの推定

- 例) XOR演算後のデータにxkeyコマンドでXORキーを推定

```
$ ef /bin/ls | xor 0xcc | peek -W 16 -r
```

```
-----  
-  
00000: B3 89 80 8A CE CD CD CC CC CC CC CC CC CC CC CC .....  
00010: CF CC F2 CC CD CC CC CC FC A1 CC CC CC CC CC CC .....  
00020: 8C CC CC CC CC CC CC CC E4 E8 CE CC CC CC CC CC .....  
00030: CC CC CC CC 8C CC F4 CC C1 CC 8C CC D3 CC D2 CC .....  
00040: CA CC CC CC C8 CC CC CC 8C CC CC CC CC CC CC CC .....  
00050: 8C CC CC CC CC CC CC CC 8C CC CC CC CC CC CC CC .....  
00060: 14 CE CC CC CC CC CC CC 14 CE CC CC CC CC CC CC .....  
00070: C4 CC CC CC CC CC CC CC CF CC CC CC C8 CC CC CC .....  
00080: D4 CF CC CC CC CC CC CC D4 CF CC CC CC CC CC CC .....  
00090: D4 CF CC CC CC CC CC CC D0 CC CC CC CC CC CC CC .....  
-----
```

```
-  
$ ef /bin/ls | xor 0xcc | xkey | hex -R  
CC
```

xkeyコマンドがXORキーを推定する方法

- XOR演算
 - NULLバイト(0x00)に演算すると、演算結果がXORキーの値になるという性質がある
- 例) NULLのみから構成される8バイトのデータにXOR演算するとXORキーの11223344AABBCCDDが演算結果にそのまま表れる

```
$ emit h:000000000000000000 | xor h:11223344AABBCCDD | peek -W 16 -r
```

```
-----  
0: 11 22 33 44 AA BB CC DD . "3D.....  
-----
```

```
0 xor 0 => 0
```

```
0 xor 1 => 1 (0x00(NULL) に XORキーを適用するとXORキーのまま)
```

```
1 xor 0 => 1
```

```
1 xor 1 => 0
```


/bin/lsの先頭部分 0x00はNULL文字

```
$ ef /bin/ls | peek -W 16 -r
```

```
-----  
00000: 7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00 .ELF.....  
00010: 03 00 3E 00 01 00 00 00 30 6D 00 00 00 00 00 00 ..>.....0m.....  
00020: 40 00 00 00 00 00 00 00 28 24 02 00 00 00 00 00 @.....($.....  
00030: 00 00 00 00 40 00 38 00 0D 00 40 00 1F 00 1E 00 ....@.8...@.....  
00040: 06 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00 .....@.....  
00050: 40 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 @.....@.....  
00060: D8 02 00 00 00 00 00 00 D8 02 00 00 00 00 00 00 .....  
00070: 08 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00 .....  
00080: 18 03 00 00 00 00 00 00 18 03 00 00 00 00 00 00 .....  
00090: 18 03 00 00 00 00 00 00 1C 00 00 00 00 00 00 00 .....  
-----
```

XORキーの推定

- 例) XOR演算後のデータにxkeyコマンドでXORキーを推定

```
$ ef /bin/ls | xor 0xcc | peek -W 16 -r
```

```
-----  
-  
00000: B3 89 80 8A CE CD CD CC CC CC CC CC CC CC CC CC .....  
00010: CF CC F2 CC CD CC CC CC FC A1 CC CC CC CC CC CC .....  
00020: 8C CC CC CC CC CC CC CC E4 E8 CE CC CC CC CC CC .....  
00030: CC CC CC CC 8C CC F4 CC C1 CC 8C CC D3 CC D2 CC .....  
00040: CA CC CC CC C8 CC CC CC 8C CC CC CC CC CC CC CC .....  
00050: 8C CC CC CC CC CC CC CC 8C CC CC CC CC CC CC CC .....  
00060: 14 CE CC CC CC CC CC CC 14 CE CC CC CC CC CC CC .....  
00070: C4 CC CC CC CC CC CC CC CF CC CC CC C8 CC CC CC .....  
00080: D4 CF CC CC CC CC CC CC D4 CF CC CC CC CC CC CC .....  
00090: D4 CF CC CC CC CC CC CC D0 CC CC CC CC CC CC CC .....  
-----
```

```
-  
$ ef /bin/ls | xor 0xcc | xkey | hex -R  
CC
```

/bin/lsの先頭部分 0x00はNULL文字

- XORキーをFFEEDDCCとすると, XOR演算結果に FFEEDDCCが多くあらわれている

```
$ ef /bin/ls | xor h:FFEEDDCC | peek -W 16 -r
```

```
-----  
00000: 80 AB 91 8A FD EF DC CC FF EE DD CC FF EE DD CC .....  
00010: FC EE E3 CC FE EE DD CC CF 83 DD CC FF EE DD CC .....  
00020: BF EE DD CC FF EE DD CC D7 CA DF CC FF EE DD CC .....  
00030: FF EE DD CC BF EE E5 CC F2 EE 9D CC E0 EE C3 CC .....  
00040: F9 EE DD CC FB EE DD CC BF EE DD CC FF EE DD CC .....  
00050: BF EE DD CC FF EE DD CC BF EE DD CC FF EE DD CC .....  
00060: 27 EC DD CC FF EE DD CC 27 EC DD CC FF EE DD CC '.....'  
00070: F7 EE DD CC FF EE DD CC FC EE DD CC FB EE DD CC .....  
00080: E7 ED DD CC FF EE DD CC E7 ED DD CC FF EE DD CC .....  
00090: E7 ED DD CC FF EE DD CC E3 EE DD CC FF EE DD CC .....  
-----
```

```
$ ef /bin/ls | xor h:FFEEDDCC | xkey | hex -R  
FFEEDDCC
```

XORキー推定時の注意事項

- 0x00(NULL)を多く含まないと推定できない
- 例) secret messageをXORキー0xccでXORした結果をxkeyでXORキーを推定するとA9と誤判定

```
$ echo "secret message" | xor 0xcc | peek -W 16 -r
```

```
-----  
0: BF A9 AF BE A9 B8 EC A1 A9 BF BF AD AB A9 C6 .....
```

```
-----  
$ echo "secret message" | xor 0xcc | xkey | hex -R
```

```
A9
```

xkeyコマンドは、2バイト目から32バイト目までの31バイト分 (Pythonスライス記法の1:32)の値の出現頻度からXORキーを推定
この範囲のデータがXORキーの推定に適していない場合は、snip
コマンドで推定に使う範囲を切り出す

```
// /bin/lsの0x1000から0x1300までの範囲を出力
```

```
$ ef /bin/ls | snip 0x1000:0x1300 | peek -W 16 -r
```

```
-----  
000: 00 5F 5F 6C 69 62 63 5F 73 74 61 72 74 5F 6D 61  .__libc_start_ma  
010: 69 6E 00 5F 5F 63 78 61 5F 66 69 6E 61 6C 69 7A  in.__cxa_finaliz  
020: 65 00 5F 5F 63 78 61 5F 61 74 65 78 69 74 00 73  e.__cxa_atexit.s  
030: 74 72 63 6D 70 00 6F 62 73 74 61 63 6B 5F 61 6C  trcmp.obstack_al  
040: 6C 6F 63 5F 66 61 69 6C 65 64 5F 68 61 6E 64 6C  loc_failed_handl  
050: 65 72 00 73 74 64 6F 75 74 00 5F 5F 6F 76 65 72  er.stdout.__over  
060: 66 6C 6F 77 00 66 77 72 69 74 65 5F 75 6E 6C 6F  flow.fwrite_unlo  
070: 63 6B 65 64 00 66 70 75 74 73 5F 75 6E 6C 6F 63  cked.fputs_unloc  
080: 6B 65 64 00 5F 5F 70 72 69 6E 74 66 5F 63 68 6B  ked.__printf_chk  
090: 00 5F 5F 6D 65 6D 70 63 70 79 5F 63 68 6B 00 6E  .__memcpy_chk.n  
-----
```

```
// /bin/lsの0x1000から0x1300までの範囲をXORキーFFEEDDCCでxor
```

```
$ ef /bin/ls | xor h:FFEEDDCC | snip 0x1000:0x1300 | peek -W 16 -r
```

```
-----  
000: FF B1 82 A0 96 8C BE 93 8C 9A BC BE 8B B1 B0 AD  ....  
010: 96 80 DD 93 A0 8D A5 AD A0 88 B4 A2 9E 82 B4 B6  ....  
020: 9A EE 82 93 9C 96 BC 93 9E 9A B8 B4 96 9A DD BF  ....  
030: 8B 9C BE A1 8F EE B2 AE 8C 9A BC AF 94 B1 BC A0  ....  
040: 93 81 BE 93 99 8F B4 A0 9A 8A 82 A4 9E 80 B9 A0  ....  
050: 9A 9C DD BF 8B 8A B2 B9 8B EE 82 93 90 98 B8 BE  ....  
060: 99 82 B2 BB FF 88 AA BE 96 9A B8 93 8A 80 B1 A3  ....  
070: 9C 85 B8 A8 FF 88 AD B9 8B 9D 82 B9 91 82 B2 AF  ....  
080: 94 8B B9 CC A0 B1 AD BE 96 80 A9 AA A0 8D B5 A7  ....  
090: FF B1 82 A1 9A 83 AD AF 8F 97 82 AF 97 85 DD A2  ....  
-----
```

```
// /bin/lsの0x1000から0x1300までの範囲をXORキーFFEEDDCCでxorしてxkeyでXORキーを推測
```

```
$ ef /bin/ls | xor h:FFEEDDCC | snip 0x1000:0x1300 | xkey | hex -R
```

```
8B9A82A3A0B1DDA9
```

```
// XORキーの推定に使う範囲として257バイトから512バイトまでを指定してXORキーを推定
$ ef /bin/ls | snip 256:512 | peek -W 16 -r
```

```
-----
000: 00 40 00 00 00 00 00 00 00 41 4F 01 00 00 00 00 00  .@.....AO.....
010: 41 4F 01 00 00 00 00 00 00 00 10 00 00 00 00 00  AO.....
020: 01 00 00 00 04 00 00 00 00 00 90 01 00 00 00 00  .....
030: 00 90 01 00 00 00 00 00 00 00 90 01 00 00 00 00  .....
040: B0 71 00 00 00 00 00 00 00 B0 71 00 00 00 00 00  .q.....q.....
050: 00 10 00 00 00 00 00 00 00 01 00 00 00 06 00 00 00  .....
060: 30 0F 02 00 00 00 00 00 00 30 1F 02 00 00 00 00 00  0.....0.....
070: 30 1F 02 00 00 00 00 00 00 48 13 00 00 00 00 00 00  0.....H.....
080: E8 25 00 00 00 00 00 00 00 00 10 00 00 00 00 00 00  .%.....
090: 02 00 00 00 06 00 00 00 38 1A 02 00 00 00 00 00 00  .....8.....
-----
```

```
$ ef /bin/ls | xor h:FFEEDDCC | snip 256:512 | peek -W 16 -r
```

```
-----
000: FF AE DD CC FF EE DD CC BE A1 DC CC FF EE DD CC  .....
010: BE A1 DC CC FF EE DD CC FF FE DD CC FF EE DD CC  .....
020: FE EE DD CC FB EE DD CC FF 7E DC CC FF EE DD CC  .....~.....
030: FF 7E DC CC FF EE DD CC FF 7E DC CC FF EE DD CC  .~.....~.....
040: 4F 9F DD CC FF EE DD CC 4F 9F DD CC FF EE DD CC  0.....0.....
050: FF FE DD CC FF EE DD CC FE EE DD CC F9 EE DD CC  .....
060: CF E1 DF CC FF EE DD CC CF F1 DF CC FF EE DD CC  .....
070: CF F1 DF CC FF EE DD CC B7 FD DD CC FF EE DD CC  .....
080: 17 CB DD CC FF EE DD CC FF FE DD CC FF EE DD CC  .....
090: FD EE DD CC F9 EE DD CC C7 F4 DF CC FF EE DD CC  .....
-----
```

```
$ ef /bin/ls | xor h:FFEEDDCC | snip 256:512 | xkey | hex -R
FFEEDDCC
```

XORキー推定による自動デコード

- autoxorコマンド
 - パイプで受けとったデータについてxkeyコマンドの機能を使ってXORキーを推定し，XOR演算結果を出力
- 例) /bin/lsファイルにXORキー0xCCでXOR演算をして，その結果のデータに対してautoxorコマンドで自動的にXORキーを推定してデコード

```
// /bin/lsの先頭部分の内容を出力
```

```
$ ef /bin/ls | peek -W 16 -r
```

```
-----
00000: 7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00 .ELF.....
00010: 03 00 3E 00 01 00 00 00 30 6D 00 00 00 00 00 00 ..>.....0m.....
00020: 40 00 00 00 00 00 00 00 28 24 02 00 00 00 00 00 @.....($.....
00030: 00 00 00 00 40 00 38 00 0D 00 40 00 1F 00 1E 00 ....@.8...@.....
00040: 06 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00 .....@.....
00050: 40 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 @.....@.....
00060: D8 02 00 00 00 00 00 00 D8 02 00 00 00 00 00 00 .....
00070: 08 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00 .....
00080: 18 03 00 00 00 00 00 00 18 03 00 00 00 00 00 00 .....
00090: 18 03 00 00 00 00 00 00 1C 00 00 00 00 00 00 00 .....
-----
```

```
// /bin/lsのmd5ハッシュ値を出力
```

```
$ ef /bin/ls | pf {md5}
```

```
1adf6740fd2848bc1cbcbad4cb97a027
```

```
// /bin/lsをXORキー0xccでXORした結果の先頭部分の内容を出力
```

```
$ ef /bin/ls | xor 0xcc | peek -W 16 -r
```

```
-----
00000: B3 89 80 8A CE CD CD CC CC CC CC CC CC CC CC .....
00010: CF CC F2 CC CD CC CC CC FC A1 CC CC CC CC CC CC .....
00020: 8C CC CC CC CC CC CC CC E4 E8 CE CC CC CC CC CC .....
00030: CC CC CC CC 8C CC F4 CC C1 CC 8C CC D3 CC D2 CC .....
00040: CA CC CC CC C8 CC CC CC 8C CC CC CC CC CC CC CC .....
00050: 8C CC CC CC CC CC CC CC 8C CC CC CC CC CC CC CC .....
00060: 14 CE CC CC CC CC CC CC 14 CE CC CC CC CC CC CC .....
00070: C4 CC CC CC CC CC CC CC CF CC CC CC C8 CC CC CC .....
00080: D4 CF CC CC CC CC CC CC D4 CF CC CC CC CC CC CC .....
00090: D4 CF CC CC CC CC CC CC D0 CC CC CC CC CC CC CC .....
-----
```


// 比較のため、前ページの内容そのまま再掲

```
$ ef /bin/ls | peek -W 16 -r
```

```
-----
00000: 7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00 .ELF.....
00010: 03 00 3E 00 01 00 00 00 30 6D 00 00 00 00 00 00 ..>.....0m.....
00020: 40 00 00 00 00 00 00 00 28 24 02 00 00 00 00 00 @.....($.....
00030: 00 00 00 00 40 00 38 00 0D 00 40 00 1F 00 1E 00 ....@.8....@.....
00040: 06 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00 .....@.....
00050: 40 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 @.....@.....
00060: D8 02 00 00 00 00 00 00 D8 02 00 00 00 00 00 00 .....
00070: 08 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00 .....
00080: 18 03 00 00 00 00 00 00 18 03 00 00 00 00 00 00 .....
00090: 18 03 00 00 00 00 00 00 1C 00 00 00 00 00 00 00 .....
-----
```

```
$ ef /bin/ls | pf {md5}
```

```
1adf6740fd2848bc1cbcbad4cb97a027
```

// /bin/lsをXORキー0xccでXORした結果をデコードし、先頭部分の内容を出力(上記と一致)

```
$ ef /bin/ls | xor 0xcc | autoxor | peek -W 16 -r
```

```
-----
00000: 7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00 .ELF.....
00010: 03 00 3E 00 01 00 00 00 30 6D 00 00 00 00 00 00 ..>.....0m.....
00020: 40 00 00 00 00 00 00 00 28 24 02 00 00 00 00 00 @.....($.....
00030: 00 00 00 00 40 00 38 00 0D 00 40 00 1F 00 1E 00 ....@.8....@.....
00040: 06 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00 .....@.....
00050: 40 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 @.....@.....
00060: D8 02 00 00 00 00 00 00 D8 02 00 00 00 00 00 00 .....
00070: 08 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00 .....
00080: 18 03 00 00 00 00 00 00 18 03 00 00 00 00 00 00 .....
00090: 18 03 00 00 00 00 00 00 1C 00 00 00 00 00 00 00 .....
-----
```

// /bin/lsをXORキー0xccでXORした結果をデコードした結果のMD5ハッシュ値を出力(一致)

```
$ ef /bin/ls | xor 0xcc | autoxor | pf {md5}
```

```
1adf6740fd2848bc1cbcbad4cb97a027
```

Binary Refinery

デコードとエンコード

- b64コマンド
- revコマンド
- byteswapコマンド
- rotコマンド
- urlコマンド

b64コマンド使用例

- b64コマンド
 - パイプで受けとったデータをBASE64でデコード
- BASE64
 - AからZの26文字, aからzの26文字, 0から9の10文字, +と/の2文字の合計64文字とパディングのための=を使ってバイナリデータとテキストデータを変換するエンコード方式
 - 電子メールの添付ファイルなど様々な用途で利用

b64コマンド使用例

```
// Binary RefineryをBASE64エンコード
```

```
$ echo "Binary Refinery" | b64 -R
```

```
QmluYXJ5IFJlZmluZXJ5Cg==
```

```
// BASE64でデコード
```

```
$ emit QmluYXJ5IFJlZmluZXJ5Cg== | b64
```

```
Binary Refinery
```

```
$ echo QmluYXJ5IFJlZmluZXJ5Cg== | b64
```

```
Binary Refinery
```

```
// 0x00112233FFEEDDCC を BASE64エンコード
```

```
$ emit h:00112233FFEEDDCC | b64 -R
```

```
ABEiM//u3cw=
```

```
// BASE64でデコード
```

```
$ emit ABEiM//u3cw= | b64 | peek -W 16 -r
```

```
-----  
0: 00 11 22 33 FF EE DD CC
```

```
.."3....  
-----
```

revコマンド使用例

- revコマンド
 - パイプで受けとったデータのバイト列を逆順に並べ替える
 - Pythonのスライスで[::-1]のようにバイト列を逆順に並べ替えることができる
 - オプション-B でブロックサイズを指定してブロックサイズ単位で逆順に並べ替え可能
- 例)
 - 00112233445566778899AABBCCDDEEFFの16バイト列を逆順に並べ替え

revコマンド使用例

```
$ emit h:00112233445566778899AABBCCDDEEFF | rev | peek -W 16 -r
```

```
-----  
00: FF EE DD CC BB AA 99 88 77 66 55 44 33 22 11 00 .....wfUD3"..  
-----
```

// Pythonでの逆順への並べ替え例

```
$ emit h:00112233445566778899AABBCCDDEEFF | python3 -c "import sys;  
data = sys.stdin.buffer.read(); sys.stdout.buffer.write(data[::-1])" | peek -W 16 -r
```

```
-----  
00: FF EE DD CC BB AA 99 88 77 66 55 44 33 22 11 00 .....wfUD3"..  
-----
```

revコマンド使用例

// 2バイト単位で逆順に並べ替え

```
$ emit h:00112233445566778899AABBCCDDEEFF | rev -B 2 | peek -W 16 -r
```

```
-----  
00: EE FF CC DD AA BB 88 99 66 77 44 55 22 33 00 11 .....fwDU"3..  
-----
```

// 4バイト単位で逆順に並べ替え

```
$ emit h:00112233445566778899AABBCCDDEEFF | rev -B 4 | peek -W 16 -r
```

```
-----  
00: CC DD EE FF 88 99 AA BB 44 55 66 77 00 11 22 33 .....DUfw.."3  
-----
```

// 8バイト単位で逆順に並べ替え

```
$ emit h:00112233445566778899AABBCCDDEEFF | rev -B 8 | peek -W 16 -r
```

```
-----  
00: 88 99 AA BB CC DD EE FF 00 11 22 33 44 55 66 77 ..... "3DUfw  
-----
```

byteswapコマンド使用例

- byteswapコマンド
 - revコマンドとは異なる方法でバイト列を並べ替える
 - revコマンドは-Bオプションで指定したブロックサイズでデータを分割してブロック単位の順序を逆順に入れ替えて、各ブロック内のバイト列の順序はそのまま
 - 引数で指定したブロックサイズでデータを分割してブロック同士の順序はそのまま変えずに、各ブロック内のバイト列の順序を逆順に入れ替える
 - デフォルトブロックサイズ: 4バイト

byteswapコマンド使用例

// ブロックサイズ(4バイト)で1ブロック内を逆順に並べ替え

```
$ emit h:00112233445566778899AABBCCDDEEFF | byteswap | peek -W 16 -r
```

```
-----  
00: 33 22 11 00 77 66 55 44 BB AA 99 88 FF EE DD CC 3"..wfUD.....  
-----
```

// ブロックサイズ(1バイト)で1ブロック内を逆順に並べ替え

```
$ emit h:00112233445566778899AABBCCDDEEFF | byteswap 1 | peek -W 16 -r
```

```
-----  
00: 00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF .."3DUfw.....  
-----
```

// ブロックサイズ(2バイト)で1ブロック内を逆順に並べ替え

```
$ emit h:00112233445566778899AABBCCDDEEFF | byteswap 2 | peek -W 16 -r
```

```
-----  
00: 11 00 33 22 55 44 77 66 99 88 BB AA DD CC FF EE ..3"UDwf.....  
-----
```

// ブロックサイズ(4バイト)で1ブロック内を逆順に並べ替え

```
$ emit h:00112233445566778899AABBCCDDEEFF | byteswap 4 | peek -W 16 -r
```

```
-----  
00: 33 22 11 00 77 66 55 44 BB AA 99 88 FF EE DD CC 3"..wfUD.....  
-----
```

// ブロックサイズ(8バイト)で1ブロック内を逆順に並べ替え

```
$ emit h:00112233445566778899AABBCCDDEEFF | byteswap 8 | peek -W 16 -r
```

```
-----  
00: 77 66 55 44 33 22 11 00 FF EE DD CC BB AA 99 88 wfUD3".....  
-----
```

revとbyteswapの注意事項

- revコマンド・byteswapコマンド
 - 指定したブロックサイズが2バイト以上でパイプで受けとったデータがブロックサイズの倍数でない場合に、データの末尾のブロックサイズに満たない部分を切り捨てる ⇒ データの欠落が生じる
- 例) 001122334455667788AABBCCDDEE (15バイト)のデータに対してブロックサイズ8バイトとしてrevとbyteswapを実行して、8899AABBCCDDEEの7バイトがブロックサイズに満たないので切り捨て

revとbyteswapの注意事項

```
$ emit h:00112233445566778899AABBCCDDEE | byteswap 8 | peek -W 16 -r
```

```
-----  
0: 77 66 55 44 33 22 11 00                                wfUD3"..  
-----
```

```
$ emit h:00112233445566778899AABBCCDDEE | rev -B 8 | peek -W 16 -r
```

```
-----  
0: 00 11 22 33 44 55 66 77                                .."3DUfw  
-----
```

シーザー暗号(rot)

- rotコマンド

- パイプで受けとったデータに含まれるアルファベットの文字を引数で指定した数の後のアルファベットの文字に置換するシーザー暗号のコマンド
- 数字や記号などのアルファベット以外の文字は置換対象外
- デフォルト: 13 (ROT13)
- シーザー暗号の詳細は省略

```
$ echo "IBM" | rot 25
```

```
HAL
```

```
$ echo "IBM" | rot -- -1 // -1だけだとオプションと解釈されるので, --を指定
```

```
HAL
```

```
$ echo "Binary Refinery" | rot
```

```
Ovanel Ersvarel
```

```
$ echo "***Binary Refinery##" | rot
```

```
**Ovanel Ersvarel##
```

URLデコード

- urlコマンド
 - ひらがな, カタカナ, 漢字などURLに直接使用できない文字を%xx(xxは16進数)形式でエスケープしたURLエンコードした文字列をデコードするためのコマンド
 - Pythonのurllibモジュールのurllib.parse.quote()関数とurllib.parse.unquote()関数で同様のことが可能
 - -Rオプションでエンコード

urlコマンド使用例

```
$ emit "https://jp.emeditor.com/general/【重要】  
emeditor-インストーラーのダウンロード導線に/" | url -  
R
```

```
https://jp.emeditor.com/general/%E3%80%90%E9%87%8D  
%E8%A6%81%E3%80%91emeditor%2D%E3%82%A4%E3%83%B3%E3  
%82%B9%E3%83%88%E3%83%BC%E3%83%A9%E3%83%BC%E3%81%A  
E%E3%83%80%E3%82%A6%E3%83%B3%E3%83%AD%E3%83%BC%E3%  
83%89%E5%B0%8E%E7%B7%9A%E3%81%AB/
```

```
$ emit  
"https://jp.emeditor.com/general/%E3%80%90%E9%87%8  
D%E8%A6%81%E3%80%91emeditor%2D%E3%82%A4%E3%83%B3%E  
3%82%B9%E3%83%88%E3%83%BC%E3%83%A9%E3%83%BC%E3%81%  
AE%E3%83%80%E3%82%A6%E3%83%B3%E3%83%AD%E3%83%BC%E3  
%83%89%E5%B0%8E%E7%B7%9A%E3%81%AB/" | url
```

```
https://jp.emeditor.com/general/【重要】emeditor-  
インストーラーのダウンロード導線に/
```

圧縮

- Binary Refineryでは24種類の圧縮アルゴリズム(ライブラリ)をサポート
- *は, 展開(decompression)のみサポート

APLib	BriefLZ	Brotli	BZip2	FastLZ	JCALG*
LZMA	LZ4*	LZF	LZG*	LZIP*	LZJB
LZNT1	LZO*	LZW*	MSZIP*	NRV2B*	NRV2D*
NRV2E*	PKWare*	QuickLZ*	SZDD*	ZLib	ZStandard

圧縮の代表的なコマンド

- z1コマンド
- lzmaコマンド
- decompressコマンド
- ZIPファイルや圧縮展開ソフトウェア7-Zipの7zファイルなどの圧縮ファイルを展開するのではなく、圧縮ファイルやPDFファイルに含まれる個別の圧縮データを展開
- Binary Refineryには圧縮ファイルを展開するためのxt7zやxtzipなどのxtで始まるコマンドが存在

zlibで圧縮されたデータの展開

- `z1`コマンド
 - Deflate圧縮アルゴリズムを実装した
zlibライブラリで圧縮されたデータを展開
 - Deflate圧縮アルゴリズム
 - ZIPファイルやgzip(GNU zip)ファイルの圧縮などで
幅広く利用
 - デフォルト動作： 展開
 - `-R`オプション時の動作： 圧縮

zlibで圧縮されたデータの展開

- emitコマンドとパイプで受けとったデータの各バイトを引数で指定された回数ずつ繰り返して出力するBinary Refineryのstretchコマンドを繰り返して16進数で11の値が96バイト分続いているデータを出力し，z1コマンドで圧縮
- z1コマンドで圧縮するとデフォルトではヘッダは付かずに，圧縮されたデータのみ出力

zlibで圧縮されたデータの展開

```
$ emit h:11 | stretch 96 | peek -W 16 -E
```

```
-----  
00.096 kB; 00.00% entropy; data  
-----
```

```
00: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
10: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
20: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
30: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
40: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
50: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
-----
```

```
// (1) z1コマンド -Rで圧縮 gzip圧縮ヘッダなし
```

(1)1314A42D0000 が
zlib圧縮データ

```
$ emit h:11 | stretch 96 | z1 -R | peek -W 16 -E
```

```
-----  
00.006 kB; 28.15% entropy; data  
-----
```

```
0: 13 14 A4 2D 00 00 .....  
-----
```

zlibで圧縮されたデータの展開

// (2) z1 コマンド -z オプションでzlibヘッダ付与

```
$ emit h:11 | stretch 96 | z1 -R -z | peek -W 16 -E
```

00.012 kB; 42.73% entropy; **zlib compressed data**

0: 78 DA 13 14 A4 2D 00 00 35 9F 06 61 x.....-..5..a

// (3) z1 コマンド -g オプションでgzip圧縮ヘッダ付与

```
$ emit h:11 | stretch 96 | z1 -R -g | peek -W 16 -E
```

00.024 kB; 40.01% entropy; **gzip compressed data**, max compression, ...

00: 1F 8B 08 00 00 00 00 00 02 03 13 14 A4 2D 00 00-..
10: EA 5F 86 E2 60 00 00 00 .._...`...

(2) z1コマンド -z で zlib ヘッダ(78DA)とzlibフッタ(359F0651)付与

(3) z1コマンド -g で gzip圧縮ファイルの出力に, 圧縮データの前に

gzipヘッダ(1F8B080000000000203)10バイトと

gzipフッタ(EA5F86E2600000)8バイト付与

zlibヘッダ付き圧縮データの構造例

オフセット	サイズ	意味	コマンド例でのデータ
0	1	圧縮方法に関する情報	78
1	1	フラグ	DA
2	6(可変)	圧縮データ	13 14 A4 2D 00 00
8	4	Adler-32 チェックサム	35 9F 06 61

gzip圧縮ファイルの構造例

オフセット	サイズ	意味	コマンド例でのデータ
0	2	シグネチャ(固定値)	1F 8B
2	1	圧縮方法	08
3	1	フラグ	00
4	4	元のファイルの更新日時	00 00 00 00
8	1	拡張フラグ	02
9	1	OS	03
10	0(可変)	オプションヘッダ	なし
10	6(可変)	圧縮データ	13 14 A4 2D 00 00
16	4	CRC32	EA 5F 86 E2
20	4	元のファイルサイズ	60 00 00 00

LZMAで圧縮されたデータの展開

- lzmaコマンド
 - LZMA圧縮アルゴリズムで圧縮されたデータを展開
 - LZMA: Lempel-Ziv-Markov chain-Algorithm
 - LZMAは7-Zipでの7z圧縮やLinuxのxz圧縮で利用
 - デフォルト展開, -Rで圧縮
- zlibで圧縮したデータと同様のデータをlzmaで圧縮・展開

lzmaでデータ圧縮

```
$ emit h:11 | stretch 96 | peek -W 16 -E
```

```
-----  
00.096 kB; 00.00% entropy; data  
-----
```

```
00: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
10: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
20: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
30: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
40: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
50: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
-----
```

```
$ emit h:11 | stretch 96 | lzma -R | peek -W 16
```

```
-----  
00.068 kB; 57.86% entropy; XZ compressed data, checksum CRC64  
-----
```

```
00: FD 37 7A 58 5A 00 00 04 E6 D6 B4 46 02 00 21 01 .7zXZ.....F..!  
10: 1C 00 00 00 10 CF 58 CC E0 00 5F 00 06 5D 00 08 .....X..._..]..  
20: EE 96 00 00 00 00 00 00 8B 10 54 B1 74 40 C8 BB .....T.t@..  
30: 00 01 22 60 53 92 86 CA 1F B6 F3 7D 01 00 00 00 .."`S.....}  
40: 00 04 59 5A .....YZ
```


様々な圧縮アルゴリズムによる 圧縮データの自動展開

- decompress コマンド
 - 圧縮されたデータを展開するコマンド
 - Binary Refineryが対応しているすべての圧縮アルゴリズムやライブラリを自動的に展開を試みる

```
$ emit h:11 | stretch 96 | z1 -R -z | decompress | peek -W 16
```

```
-----  
00.096 kB; 00.00% entropy; data  
-----
```

```
00: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
...:                                4 repetitions
```

```
50: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
-----
```

```
$ emit h:11 | stretch 96 | z1 -R -g | decompress | peek -W 16
```

```
-----  
00.096 kB; 00.00% entropy; data  
-----
```

```
00: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
...:                                4 repetitions
```

```
50: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....  
-----
```

様々な圧縮アルゴリズムによる 圧縮データの自動展開

- decompress コマンド
 - 圧縮されたデータを展開するコマンド
 - Binary Refineryが対応しているすべての圧縮アルゴリズムやライブラリを自動的に展開を試みる

```
$ emit h:11 | stretch 96 | lzma -R | decompress | peek -W 16
-----
00.096 kB; 00.00% entropy; data
-----
00: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....
...:                                4 repetitions
50: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 .....
-----
```

演習： 難読化された PHPスクリプトの解析

- 難読化されたプログラムをBinary Refineryで解析
- 難読化(obfuscation)
 - コンピュータプログラムの動作を変えずに、プログラムコードの内部的な手続き(サブルーチン)の内容・構造・データ等を人間にとって読み取り苦肉刷るように改変・加工すること
 - 難読化の対象は、ソースコードや、ソースコードから生成される機械語またはバイトコードなどの中間表現である
 - 難読化されたコード(obfuscated code)は第三者によるプログラムの解読・解析が困難

難読化したコードの用途例

- 攻撃者がWebサーバに侵入した際、Web Shell(Webアプリケーションのプログラム)を仕込み、攻撃者がWebサーバを遠隔操作できるようにする
- Webサーバの管理者にWeb Shellが発見されてもどのような機能を有しているかを隠すために攻撃者がWeb Shellのコードを難読化することがある

演習： 難読化された PHPスクリプトの解析

- Webアプリケーションを作成するためによく利用されているPHP(PHP: Hypertext Preprocessor)で書かれた無害なプログラムを難読化したプログラムを解析
- 難読化の方式
 - ある Web Shell で実際に利用されていた方法

/home/security/binref/obfuscated.php

<?php

```
eval(gzinflate(base64_decode(strrev(str_rot13(
"==NN9U624/57W8J1whjN21027Ua78oBMpUf942Xclel
X0dnMzGWu0oA/tII40ZZChvKDcBmluAG4wt4wWDoyYqCl
nfaN5NTWVSk/+XLP0MUrp1nwxorLBH7dXc80V9QSu6isc
l+DrYrNr1jIWMx94eq3z8hJ49InAhQwF3v3F8nRrXk9eK
FBc6jn5RPMVC90p86NbpLXV0xntNcej2LZpwgR4fFaJX4
1F0QNUKqsugGM8z7nsxLTY17EHoCE37F5h+Vq41Gk3XQi
XMdsiF4uRhZkYujKV8PPUTy3tDLwlgp9ymeFf6svBx4W4
3WEEJkRuovjQ91ECI8Y9evM1oMEwSFW4B5iWzEsU4RUvf
Sc2YpTvMWfvIfc4FCEh+XmLyZmxSfEBCegXo0Pz9BbFb0
sMi9LHx7t9/s+mwwUpyhRnmp0icThz/EGXNkvFa1QjdVX
+BZFYWxkaSLRFAev1htyeCTrQN97WrgLP06vBz19isZSO
WtFqQMM" )))) )
```

?>

obfuscated.phpの実行

- 下記要領で実行するとパスワードの入力を求められる

```
// binrefディレクトリに移動
```

```
$ php obfuscated.php
```

Password: (適当に入力) (※エコーバックしない)

残念!パスワードが違います。

- 正しいパスワードを特定するのが演習のゴール

/home/security/binref/obfuscated.php

<?php

```
eval(gzinflate(base64_decode(strrev(str_rot13(
("==NN9U624/57W8J1whjN21027Ua78oBMpUf942Xclel
X0dnMzGwu0oA/tII40ZZChvKDCBmluAG4wt4wWDoyYqCl
nfaN5NTWVSk/+XLP0MUrp1nwxorLBH7dXc80V9QSu6isc
l+DrYrNr1jIWMx94eq3z8hJ49InAhQwF3v3F8nRrXk9eK
FBc6jn5RPMVC90p86NbpLXV0xntNcej2LZpwgR4fFaJX4
1F0QNUKqsugGM8z7nsxLTY17EHoCE37F5h+Vq41Gk3XQi
XMdsiF4uRhZkYujKV8PPUTy3tDLwignp9ymeFf6svBx4W4
3WEEJkRuovjQ91ECI8Y9evMloMEwSFW4B5iWzEsU4RUvf
Sc2YpTvMWfvIfc4FCEh+XmLyZmxSfEBCegXo0Pz9BbFb0
sMi9LHx7t9/s+mwwUpyhRnmp0icThz/EGXNkvFa1QjdVX
+BZFYWxkaSLRFAev1htyeCTrQN97WrgLP06vBz19isZSO
WtFqQMM"))))
```

?>

青字が難読化した箇所で，赤字が難読化のために使用した関数104

可読化方法の解明

- 難読化されたコードを以下の関数を順番に適用している

- 赤字部分

- `str_rot13()`関数
- `strrev()`関数
- `base64_decode()`関数
- `gzinflate()`関数
- `eval()`関数

```
<?php
eval(gzinflate(base64_decode(strrev(str_rot13
("==NN9U624/57W8J1whjN21027Ua78oBMpUf942Xclel
XOdnMzGwu0oA/tII4OZZChvKDcBmluAG4wt4wWDoyYqCl
nfaN5NTWVSk/+XLP0MUrplnw xorLBH7dXc80V9QSu6isc
l+DrYrNr1jIWMx94eq3z8hJ49InAhQwF3v3F8nRrXk9eK
FBc6jn5RPMVC90p86NbpLXVOxntNcej2LZpwgR4fFaJX4
1F0QNUKqsugGM8z7nsxLTY17EHOCE37F5h+Vq41Gk3XQi
XMdsiF4uRhZkYujKV8PPUTy3tDLwignp9ymeFf6svBx4W4
3WEEJkRuovjQ91ECI8Y9evMloMEwSFW4B5iWzEsU4RUvf
Sc2YpTvMWfvIfc4FCEh+XmLyZmxSfEBCegXo0Pz9BbFb0
sMi9LHx7t9/s+mWwUpyhRnmp0icThz/EGXNkvFalQjdVX
+BZFYWxkaSLRFAev1htyeCTrQN97WrgLP06vBzl9isZS0
WtFqQMM")))))
?>
```

PHP各関数の詳細はドキュメントを確認

- `str_rot13()`関数
 - <https://www.php.net/manual/ja/function.str-rot13.php>
- `strrev()`関数
 - <https://www.php.net/manual/ja/function.strrev.php>
- `base64_decode()`関数
 - <https://www.php.net/manual/ja/function.base64-decode.php>
- `gzinflate()`関数
 - <https://www.php.net/manual/ja/function.gzinflate.php>
- `eval()`関数
 - <https://www.php.net/manual/ja/function.eval.php>

PHP各関数とBinary Refineryコマンドの対応関係は？

- `str_rot13()`関数
 - Binary Refineryコマンド:
- `strrev()`関数
 - Binary Refineryコマンド:
- `base64_decode()`関数
 - Binary Refineryコマンド:
- `gzinflate()`関数
 - Binary Refineryコマンド:

難読された部分を可読化するPythonスクリプト(deobfuscate.py)を作成(1/3)

```
import sys
from refinery.units.blockwise import rev
from refinery.units.compression import zl
from refinery.units.crypto.cipher import rot
from refinery.units.encoding import b64
# 難読化されたPHPコード(青字)をbytesとしてobfuscatedに代入
obfuscated = b"""
```

```
# ROT13のデコード
rot = # ここを書く
output = rot.process(obfuscated)
```

```
<?php
eval(gzinflate(base64_decode(strrev(str_rot13(
"==NN9U624/57W8J1whjN21027Ua78oBMpUf942Xcle1
X0dnMzGwu0oA/tII4OZZChvKDCBmluAG4wt4wWDoyYqCl
nfaN5NTWVSk/+XLP0MURp1nwxorLBH7dXc80V9QSu6isc
l+DrYrNr1jIWMx94eq3z8hJ49InAhQwF3v3F8nRrXk9eK
FBc6jn5RPMVC90p86NbpLXV0xntNcej2LZpwgR4fFaJX4
1F0QNUKqsugGM8z7nsxLTY17EHoCE37F5h+Vq41Gk3XQi
XMdsiF4uRhZkYujKV8PPUTy3tDLwign9ymeFf6svBx4W4
3WEEJkRuovjQ91ECI8Y9evMloMEwSFW4B5iWzEsU4RUvf
Sc2YpTvMWfVIfc4FCEh+XmLyZmxSfEBCegXo0Pz9BbFb0
sMi9LHx7t9/s+mwwUpyhRnmp0icThz/EGXNkvFalQjdVX
+BZFYWxkaSLRFAev1htyeCTrQN97WrgLP06vBz19isZSO
WtFqQMM")))))
?>
```

難読された部分を可読化するPythonスクリプト(deobfuscate.py)を作成(2/3)

文字列の順序の反転

rev = # ここを書く

output = rev.process(output)

Base64のデコード

b64 = # ここを書く

output = b64.process(output)

Deflateアルゴリズムでの展開(zlibヘッダー無し)

z1 = # ここを書く

gen = z1.process(output)

難読された部分を可読化するPythonスクリプト(deobfuscate.py)を作成(3/3)

```
# ジェネレータgenをforループで処理する。
# 可読化されたPHPコードはbytesとなっているため、
# print()の代わりにsys.stdout.buffer.write()で出力する。
for output in gen:
    sys.stdout.buffer.write(output)
```

実行結果を確認

```
$ python3 deobfuscate.py
```

(参考)~/binref/py/br01.py

- efコマンドをPythonモジュールとして利用したスクリプト
- ファイルのパス, サイズ, 拡張子, 自動判別した種類の情報とファイルの内容についてテキストファイルの場合は先頭32文字, バイナリファイルの場合は先頭から16バイトを16進数に変換して表示

(参考)~/binref/py/br02.py

- dumpコマンドの機能を用いたPythonスクリプト
 - emitコマンドの内部で用いている
refinery.lib.argformatsモジュールの
multibin()関数でデコード可能
- multibin()関数でデコード, Python標準ライブラリのbinasciiモジュール, urllib.parseモジュール, base64モジュールでデコードするPythonスクリプト

(参考)~/binref/py/br03.py

- dumpコマンドをPythonモジュールとして利用したスクリプト
- refinery.lib.metaクラスのmetavars()関数を用いて、pfコマンドの出力に埋め込むことができるデータのサイズ、拡張子、ハッシュ値等の値を得ることができる

(参考)~/binref/py/br04.py

- peekコマンドをPythonモジュールとして利用したスクリプト
- peekクラスを用いることでデータを16進数と文字の両方で表示できるので、バイナリファイルを解析するPythonスクリプトの中で、途中のデータを表示してデバッグに利用可能

(参考)~/binref/py/br05.py

- snipコマンドとchopコマンドをPythonモジュールとして利用したデータ切り出しスクリプト
- 文字列0123456789ABCDEF
 - バイト列0x300x31...0x390x410x42...0x46
- snipクラスを用いて切り出し
- chopクラスで4バイトずつ分割

(参考)~/binref/py/br06.py

- packコマンドをPythonモジュールとして利用したスクリプト
- hexクラスを使わずに16進数のエンコード・デコードする

(参考)~/binref/py/br07.py

- addコマンドとsubコマンドをPythonモジュールとして利用したスクリプト
- バイト列への加算・減算, 複数バイト列への加算・減算の例

(参考)~/binref/py/br08.py

- neg, rotl, rotr, shl, shrコマンドを
Pythonモジュールとして利用したスクリプト
- バイト列のビット反転, ローテート, シフト

(参考)~/binref/py/br09.py

- xorコマンドとrot1コマンドをPythonモジュールとして利用したスクリプト
- パイプで受けとったデータをxorコマンドとrot1コマンドの機能を組み合わせてエンコード・デコードするPythonプログラム

(参考)~/binref/py/br10.py

- xkeyコマンドをPythonモジュールとして利用したスクリプト
- br09.pyでエンコードしたデータを自動的にデコードできるPythonスクリプト

(参考)~/binref/py/br11.py

- rev, rot, urlコマンドをPythonモジュールとして利用したスクリプト
- rev, rot, urlコマンドを用いて秘密の文字列をデコードするPythonqプログラム

(参考)~/binref/py/br12.py

- decompressコマンドをPythonモジュールとして利用したスクリプト
- 圧縮されたファイルを展開

```
$ python3 br12.py hoge.gz
```